method would return, because the class method doesn't know about singleton classes. For example, obj.class would return D, even if the class of obj is actually its singleton class, #obj.

This diagram doesn't include modules. If you're a completist, you can draw the Kernel module between Object and BasicObject. On the other hand, you probably don't want to include #Kernel in this diagram. Although modules can have singleton classes like any other object, the singleton class of Kernel is not part of obj's or #D's ancestor chains.

Apparently, Ruby organizes classes, singleton classes, and superclasses in a very purposeful pattern. The superclass of #D is #C, which is also the singleton class of C. By the same rule, the superclass of #C is #Object. Bill tries to sum it all up, making things even more confusing: "The superclass of the singleton class is the singleton class of the superclass. It's easy."

This complicated arrangement of classes, superclasses, and singleton classes can be baffling. Why does Ruby go to such lengths to organize the object model this way? The reason is that thanks to this arrangement, you can call a class method on a subclass:

```
D.a_class_method          # => "C.a_class_method()"
```

Even if a_class_method is defined on C, you can also call it on D. This is probably what you expect, but it's only possible because method lookup starts in #D and goes up to #D's superclass #C, where it finds the method.

Ingenious, isn't it? Now you can finally grasp the entire object model.

The Great Unified Theory

"The Ruby object model is a beautiful place," Bill notes, with a dreamy expression on his face. "There are classes, singleton classes, and modules. There are instance methods, class methods, and Singleton Methods."

At first glance, it all looks very complex. Look closer, and the complexity fades away. If you put singleton classes together with regular classes and modules, you end up with the seven rules of the Ruby object model:

1. There is only one kind of object—be it a regular object or a module.
2. There is only one kind of module—be it a regular module, a class, or a singleton class.
3. There is only one kind of method, and it lives in a module—most often in a class.